Nested Lists and Loops

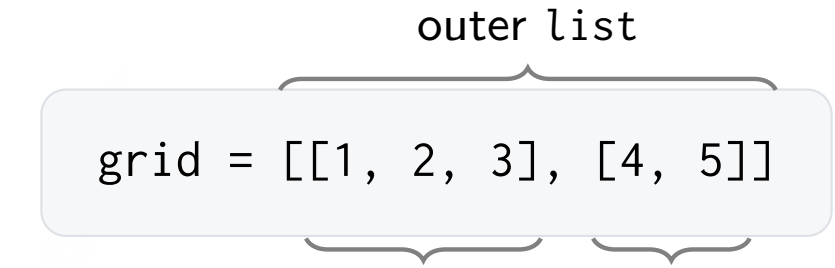
Introduction to Computer Programming

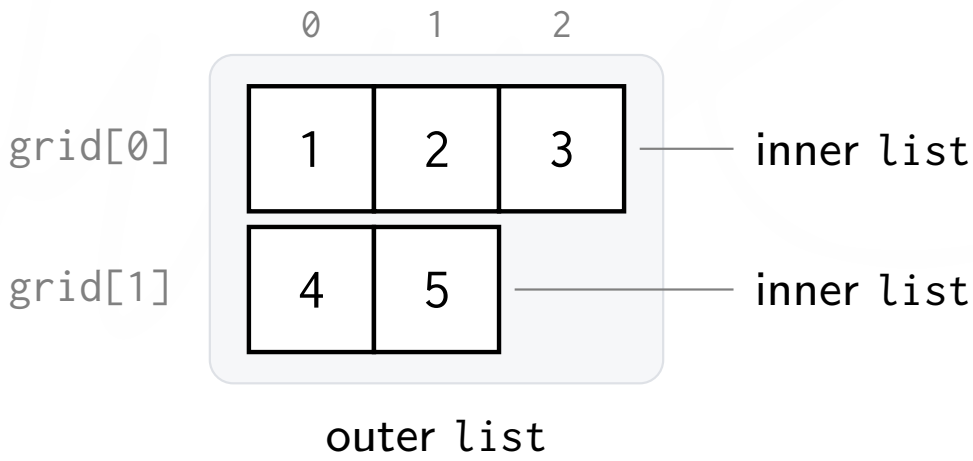
Chandra Gummaluru

chandra-gummaluru.github.io



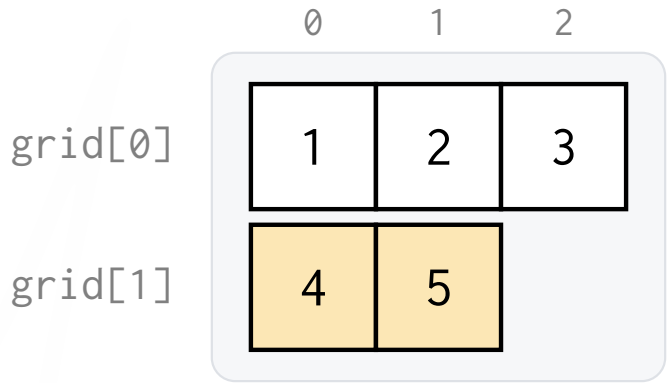
A `list` can contain other `lists`. The `list` that contains them is the **outer list** (or **nested list**); the `lists` inside are **inner `lists`** (or sub-lists).





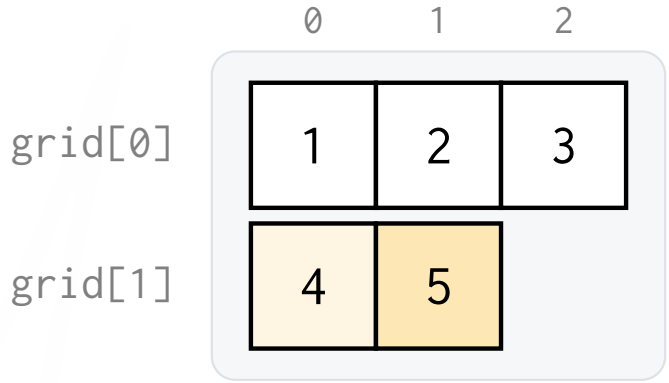
The inner `lists` don't need to have the same length.

Each inner `list` is one item of the outer `list`, so one index picks an **inner `list`**.



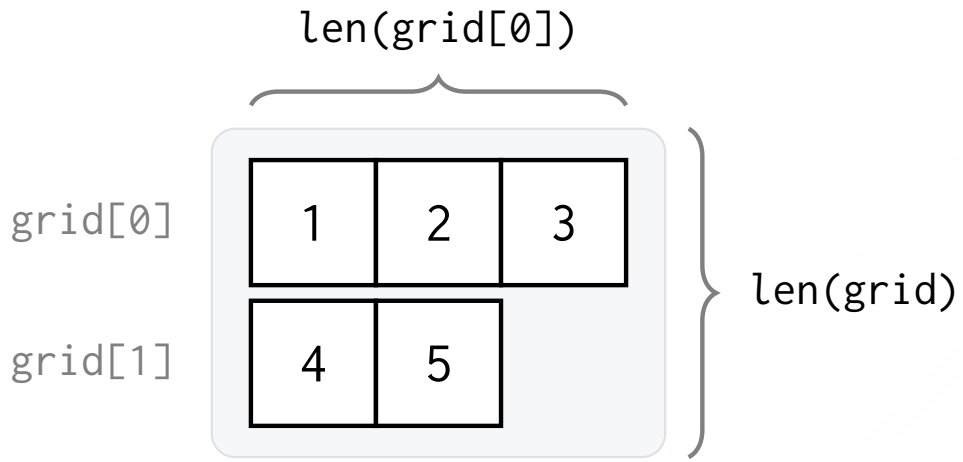

```
>>> grid = [[1, 2, 3], [4, 5]]
>>> grid[1]
[4, 5]
```

A second index picks an **item** in that inner `list`.




```
>>> grid = [[1, 2, 3], [4, 5]]
>>> grid[1][1]
5
```

`len` of the outer `list` counts its inner `lists`; `len` of an inner `list` counts its items.




```
>>> grid = [[1, 2, 3], [4, 5]]
>>> len(grid)
2
>>> len(grid[0])
3
```

64 responses

Which of these expressions will **not** evaluate to 5 ?

```
1  baskets = [['apple', 'mango', 'pear'], ['kiwi'], ['lime', 'plum']]
```

A	len(baskets[0]) + len(baskets[-1])	**6%** 4 votes
B	len(baskets[0] + baskets[2])	**5%** 3 votes
	len(baskets) - 1	**84%** 54 votes
D	len(baskets[0][1])	**5%** 3 votes

Write an expression that uses `data` and evaluates to `3`. Try to find three different ones.

```
data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
```

len(data[0][0]) = 3

data[1][0] = 3

len(data[-1][0]) = 3

len(data[-1]) = 3

len(data) = 3

Instead of parallel `lists`, the values that belong together can be kept in one **inner `list`** each.

parallel `lists`

```
bases = [10, 5, 7.5]
heights = [8, 5, 7.5]
```

one inner `list` per triangle

```
triangles = [[10, 8], [5, 5], [7.5, 7.5]]
```

`[base, height]`

Every inner `list` has the same layout, so the second index says which value it is.

An outer `list` holds the **addresses** of its inner `lists`, so another variable can refer to the **same** inner `list`.

```
>>> groceries = [['milk', 'eggs'], ['apples']]
>>> dairy = groceries[0]
>>> dairy.append('cheese')
>>> groceries
[['milk', 'eggs', 'cheese'], ['apples']]
```

`dairy` and `groceries[0]` refer to the same inner `list`

A change through either one shows up in both.

Assigning to an index of the outer `list` puts a **new** inner `list` there. Other variables still refer to the old one.

```
>>> groceries = [['milk', 'eggs'], ['apples']]
>>> dairy = groceries[0]
>>> dairy.append('cheese')
>>> groceries
[['milk', 'eggs', 'cheese'], ['apples']]
>>> groceries[0] = ['yogurt']
>>> groceries
[['yogurt'], ['apples']]
>>> dairy
['milk', 'eggs', 'cheese']
```

`groceries[0]` is a new inner `list`; `dairy` still refers to the old one

Python Workspace

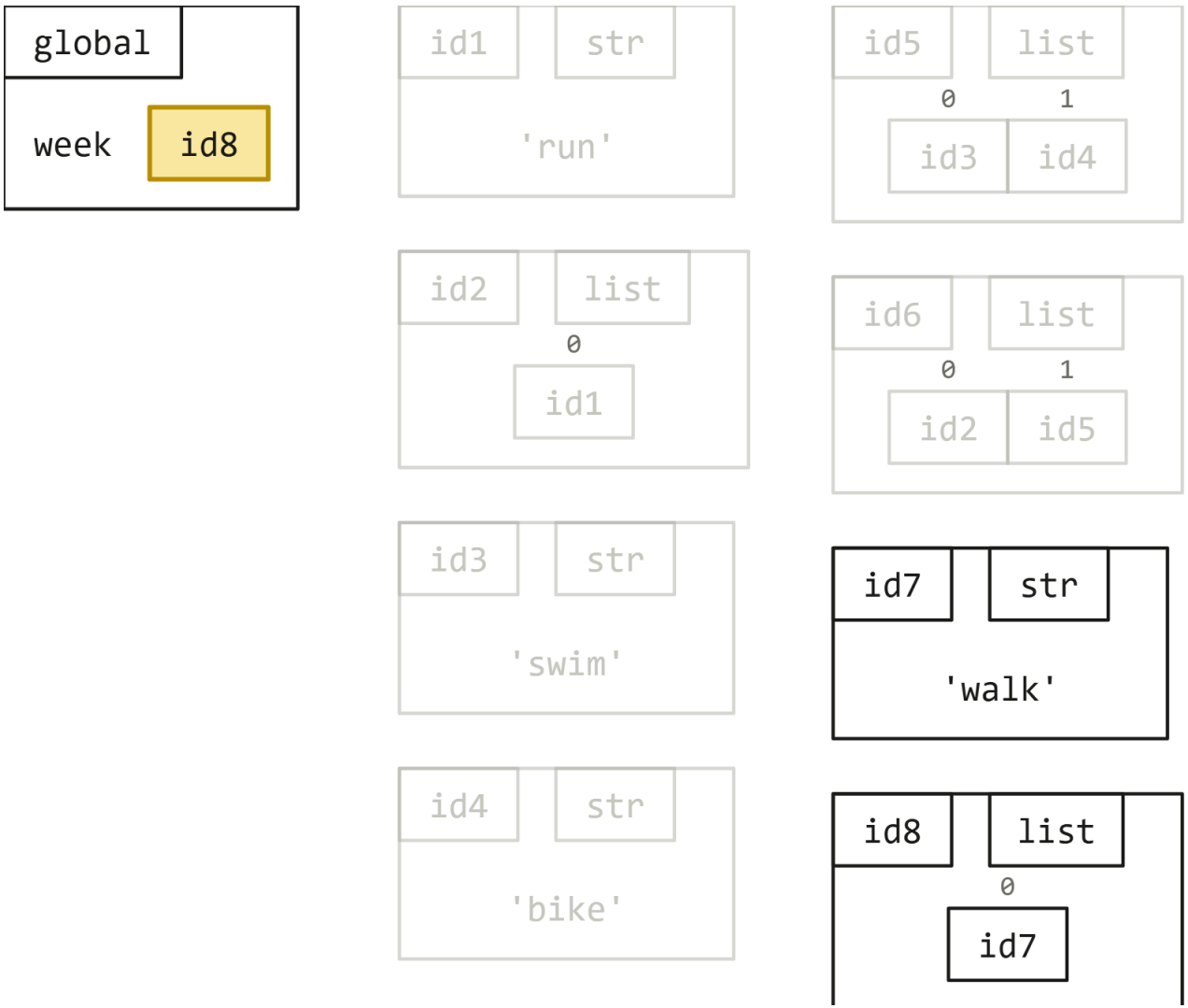
Nothing to show

nested.py

Program

```
1  week = [['run'], ['swim', 'bike']]
2
3
4  week = ['walk']
```

Memory model (step 2)


global
week
id8
id1
str
'run'
id5
list
0
1
id3
id4
id2
list
0
id1
id6
list
0
1
id2
id5
id3
str
'swim'
id7
str
'walk'
id4
str
'bike'
id8
list
0
id7

81 responses

Which groups of statements modify `data` to refer to `[['dog'], [3, 4], ['cat', 'dog']]` without changing the memory address `data` contains?

(a) `sublist = ['cat', 'dog']`

(b) `data[2] = ['cat', 'dog']`

(c) `sublist.pop()`; `sublist.pop()`; `sublist.append('dog')`

(d) `data[2].pop()`; `data[2].remove('mouse')`; `data[2].insert(1, 'dog')`

(e) `data[2] = ['cat', data[0]]`

(f) `data = [['dog'], [3, 4], ['cat', 'dog']]`

(g) `data[2] = ['cat', data[0][0]]`

(h) `data[2] = ['cat']`; `sublist.append('dog')`

```
data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
sublist = data[2]
```

A	(b), (c), (d) and (g)	**51%**	41 votes
B	(b), (d) and (g) only	**17%**	14 votes
C	(b), (c), (d), (e) and (g)	**28%**	23 votes
D	(b), (c), (d), (f) and (g)	**4%**	3 votes

main.py

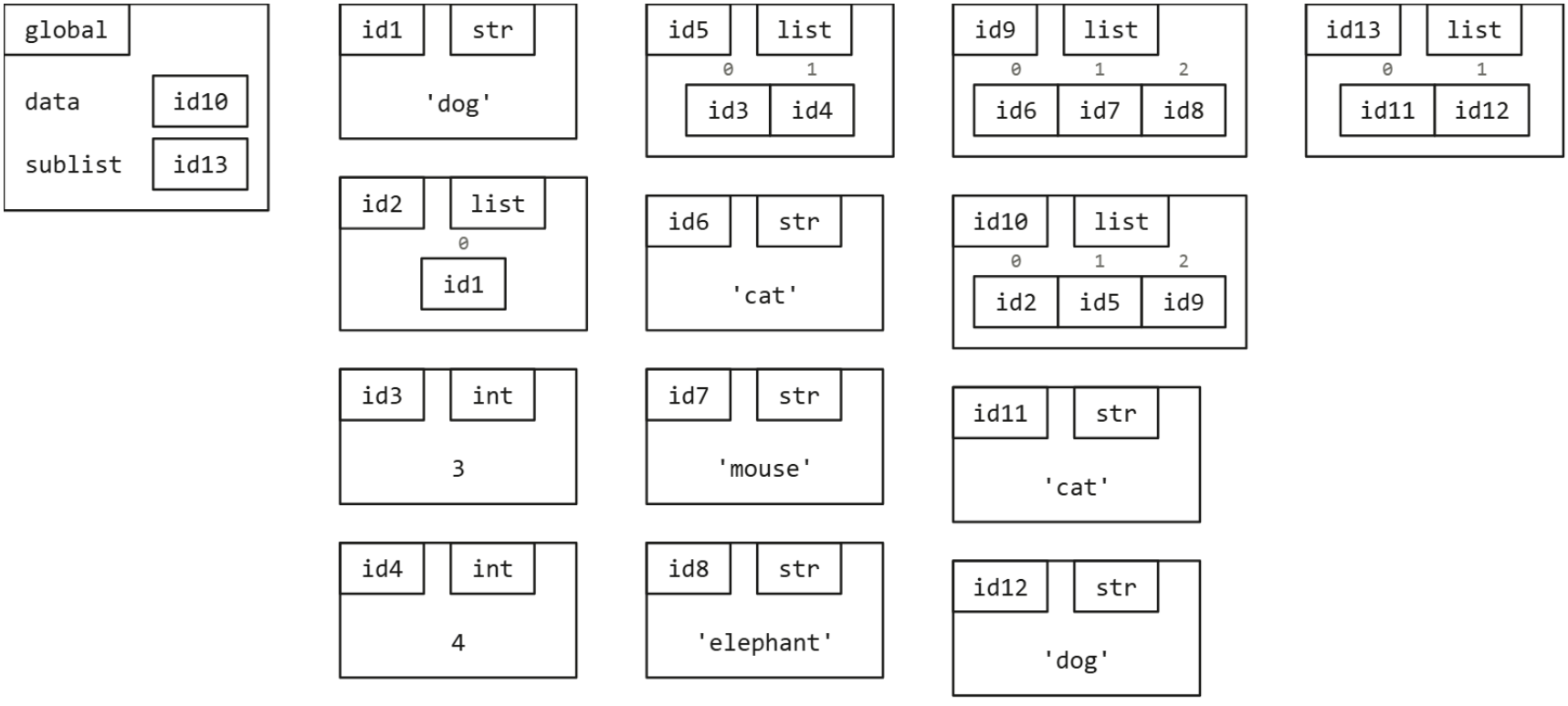
Program

```
1  data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
2  sublist = data[2]
3
4  sublist = ['cat', 'dog']
5  print(data)
```

Shell

```
Python 3.11.3
```

Memory model (step 4)


global
data
id10
sublist
id13
id1
str
'dog'
id2
list
0
id1
id3
int
3
id4
int
4
id5
list
0
1
id3
id4
id6
str
'cat'
id7
str
'mouse'
id8
str
'elephant'
id9
list
0
1
2
id6
id7
id8
id10
list
0
1
2
id2
id5
id9
id11
str
'cat'
id12
str
'dog'
id13
list
0
1
id11
id12

main.py

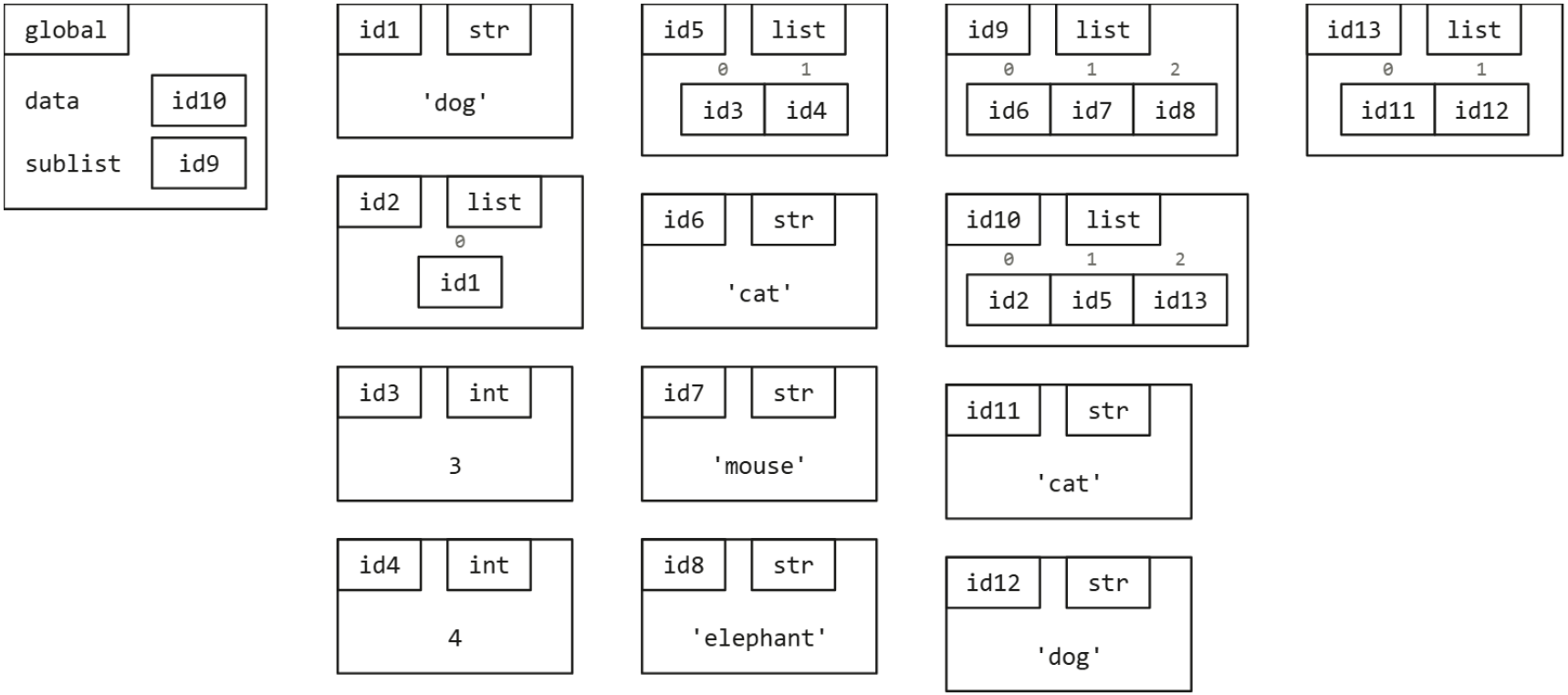
Program

```
1  data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
2  sublist = data[2]
3
4  data[2] = ['cat', 'dog']
5
6  print(data)
```

Shell

```
Python 3.11.3
```

Memory model (step 4)


global
data
id10
sublist
id9
id1
str
'dog'
id2
list
0
id1
id3
int
3
id4
int
4
id5
list
0
1
id3
id4
id6
str
'cat'
id7
str
'mouse'
id8
str
'elephant'
id9
list
0
1
2
id6
id7
id8
id10
list
0
1
2
id2
id5
id13
id11
str
'cat'
id12
str
'dog'
id13
list
0
1
id11
id12

main.py

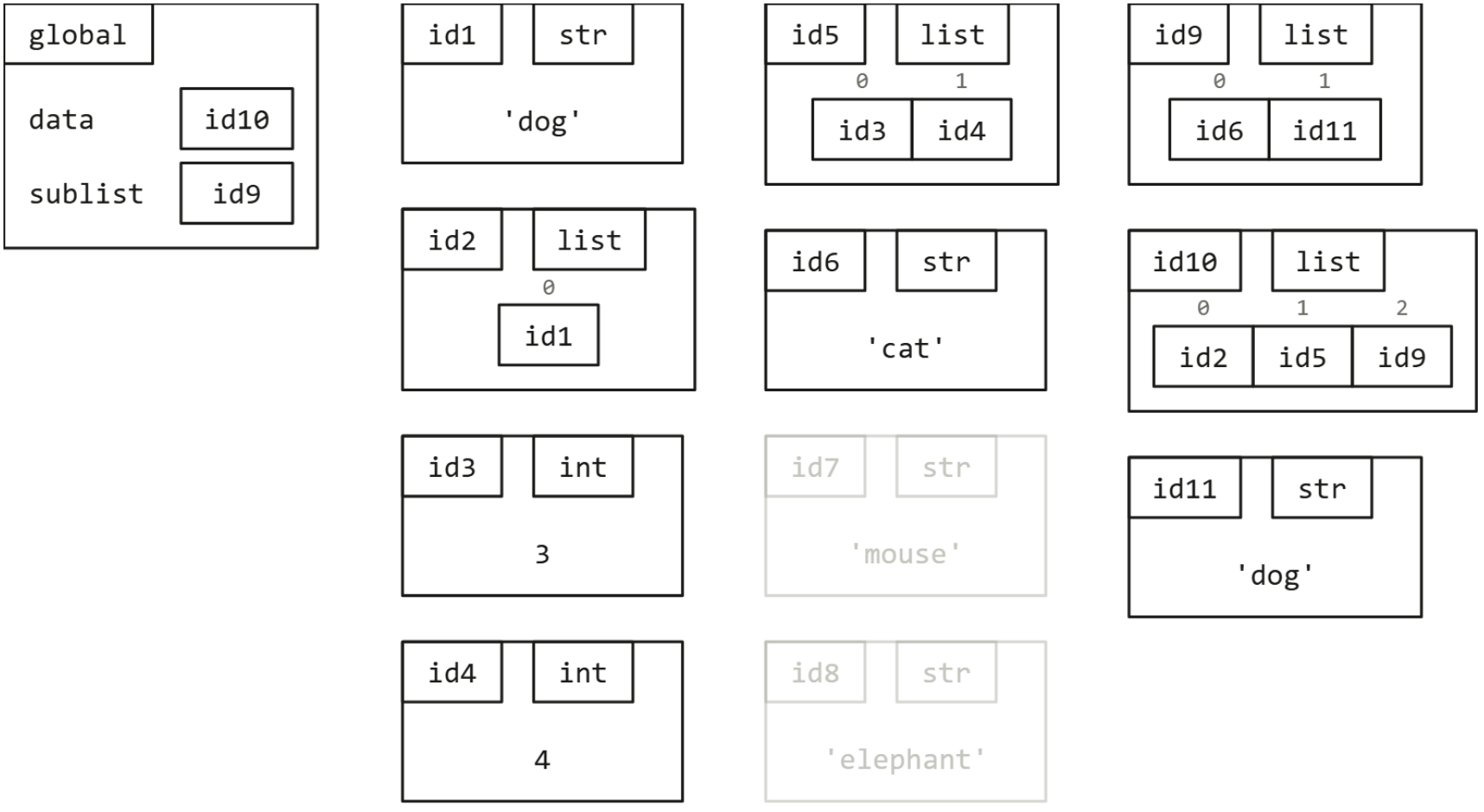
Program

```
1  data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
2  sublist = data[2]
3
4  sublist.pop()
5  sublist.pop()
6  sublist.append('dog')
7
8  print(data)
```

Shell

```
Python 3.11.3
```

Memory model (step 6)


global
data
id10
sublist
id9
id1
str
'dog'
id2
list
0
id1
id3
int
3
id4
int
4
id5
list
0
1
id3
id4
id6
str
'cat'
id7
str
'mouse'
id8
str
'elephant'
id9
list
0
1
id6
id11
id10
list
0
1
2
id2
id5
id9
id11
str
'dog'

main.py

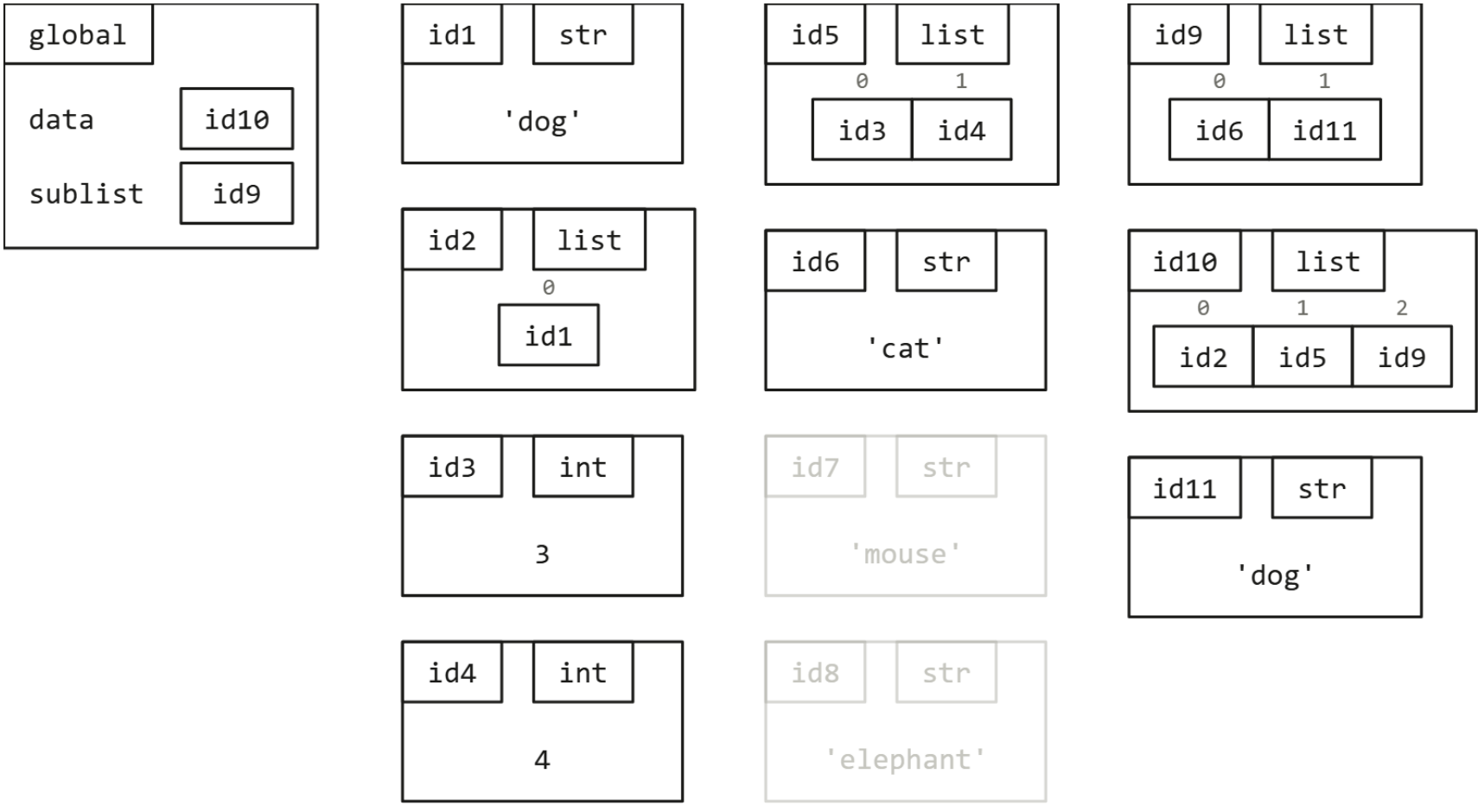
Program

```
1  data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
2  sublist = data[2]
3
4  data[2].pop()
5  data[2].remove('mouse')
6  data[2].insert(1, 'dog')
7
8  print(data)
```

Shell

```
Python 3.11.3
```

Memory model (step 6)


global
data
id10
sublist
id9
id1
str
'dog'
id2
list
0
id1
id3
int
3
id4
int
4
id5
list
0
1
id3
id4
id6
str
'cat'
id7
str
'mouse'
id8
str
'elephant'
id9
list
0
1
id6
id11
id10
list
0
1
2
id2
id5
id9
id11
str
'dog'

main.py

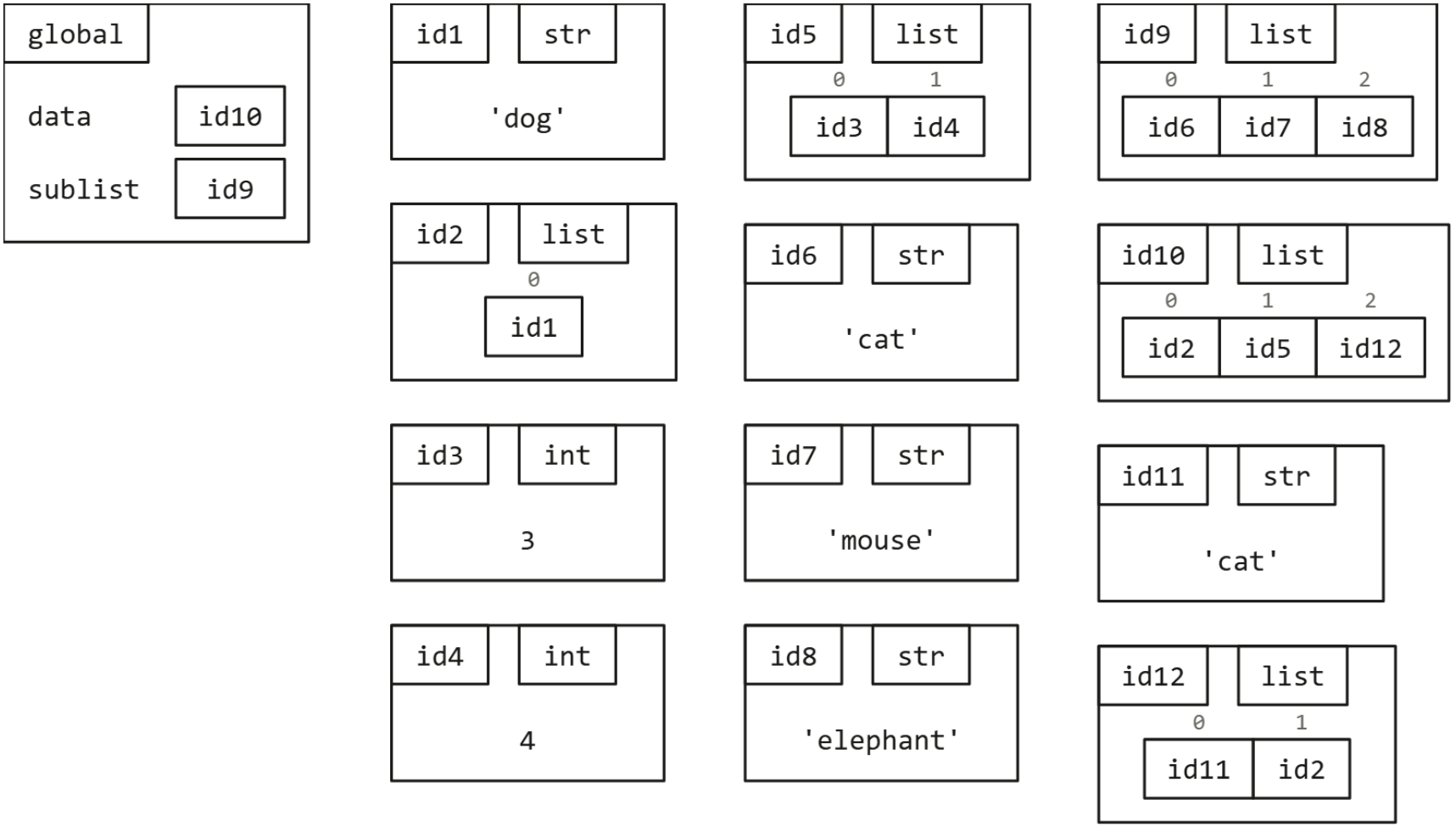
Program

```
1  data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
2  sublist = data[2]
3
4  data[2] = ['cat', data[0]]
5
6  print(data)
```

Shell

```
Python 3.11.3
```

Memory model (step 4)


global
data
id10
sublist
id9
id1
str
'dog'
id2
list
0
id1
id3
int
3
id4
int
4
id5
list
0
1
id3
id4
id6
str
'cat'
id7
str
'mouse'
id8
str
'elephant'
id9
list
0
1
2
id6
id7
id8
id10
list
0
1
2
id2
id5
id12
id11
str
'cat'
id12
list
0
1
id11
id2

main.py

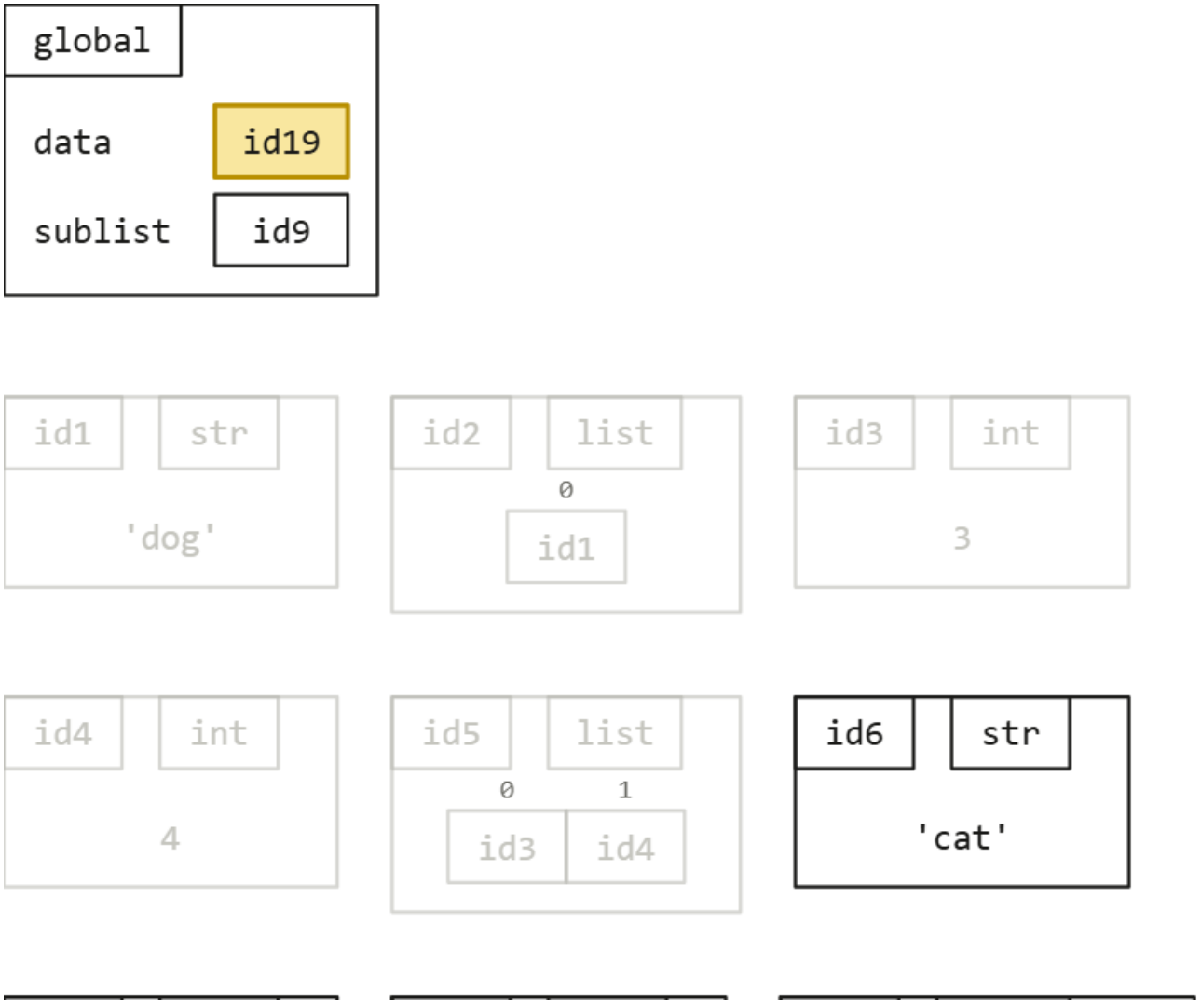
Program

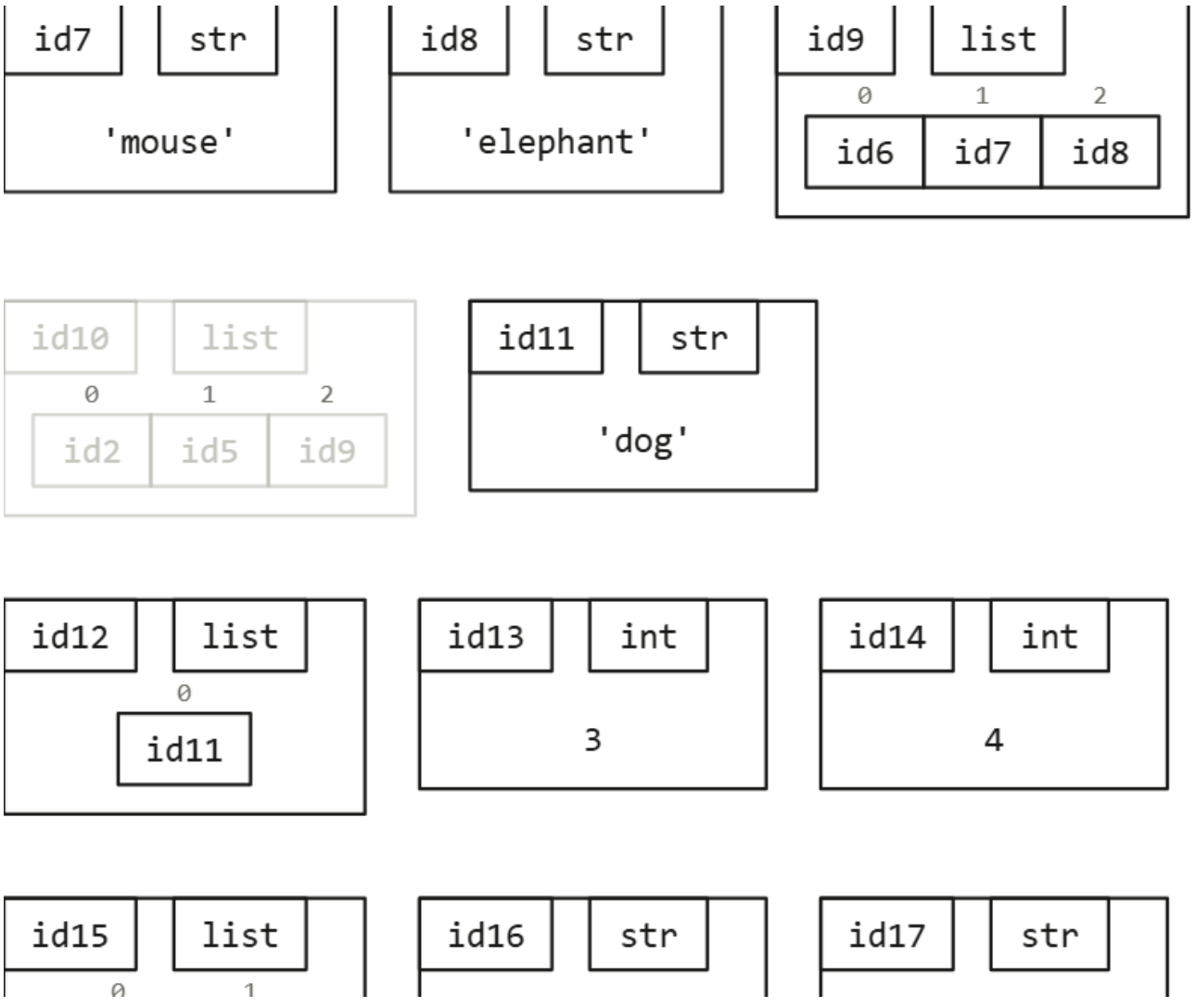
```
1  data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
2  sublist = data[2]
3
4  data = [['dog'], [3, 4], ['cat', 'dog']]
```

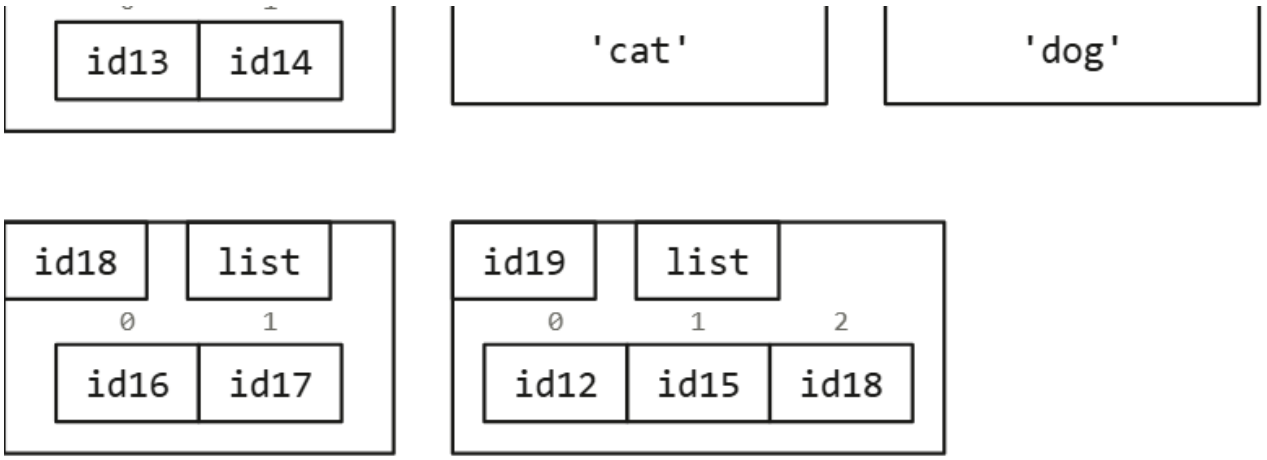
Shell

```
Python 3.11.3
```

Memory model (step 3)


global
data
id19
sublist
id9
id1
str
'dog'
id2
list
0
id1
id3
int
3
id4
int
4
id5
list
0
1
id3
id4
id6
str
'cat'


id7
str
'mouse'
id8
str
'elephant'
id9
list
0
1
2
id6
id7
id8
id10
list
0
1
2
id2
id5
id9
id11
str
'dog'
id12
list
0
id11
id13
int
3
id14
int
4
id15
list
0
1
id16
str
id17
str


id13
id14
'cat'
'dog'
id18
list
0
1
id16
id17
id19
list
0
1
2
id12
id15
id18

main.py

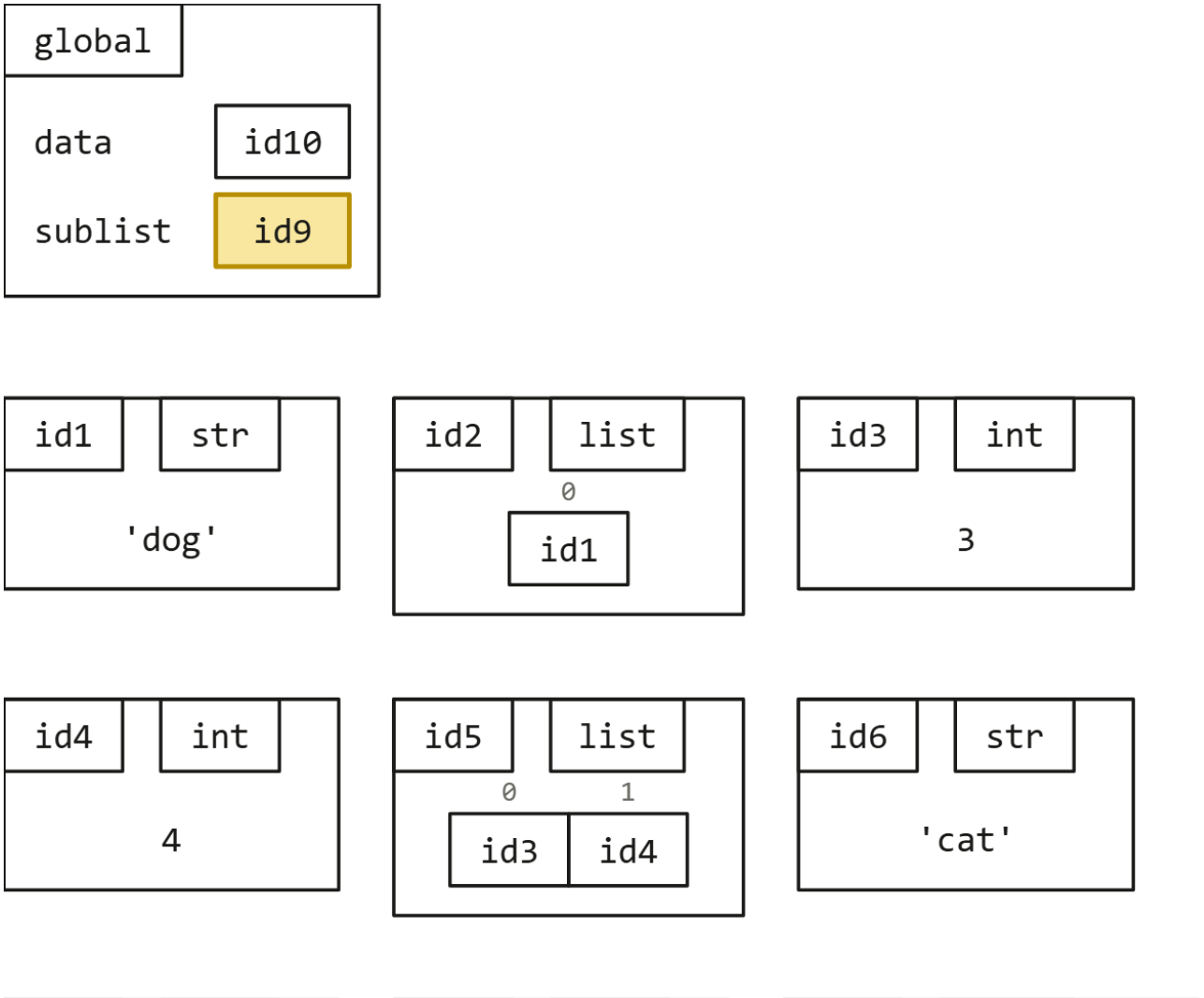
Program

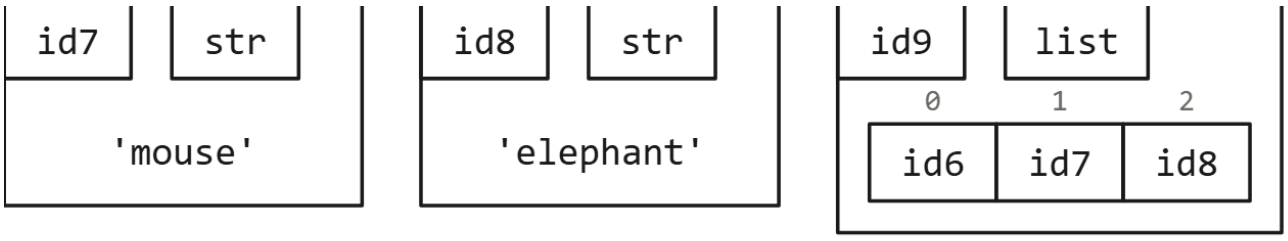
```
1  data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
2  sublist = data[2]
```

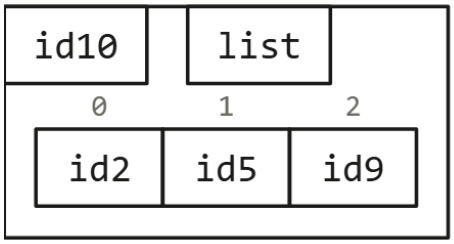
Shell

```
Python 3.11.3
```

Memory model (step 2)


global
data
id10
sublist
id9
id1
str
'dog'
id2
list
0
id1
id3
int
3
id4
int
4
id5
list
0
1
id3
id4
id6
str
'cat'


id7
str
'mouse'
id8
str
'elephant'
id9
list
0
1
2
id6
id7
id8


id10
list
0
1
2
id2
id5
id9

main.py

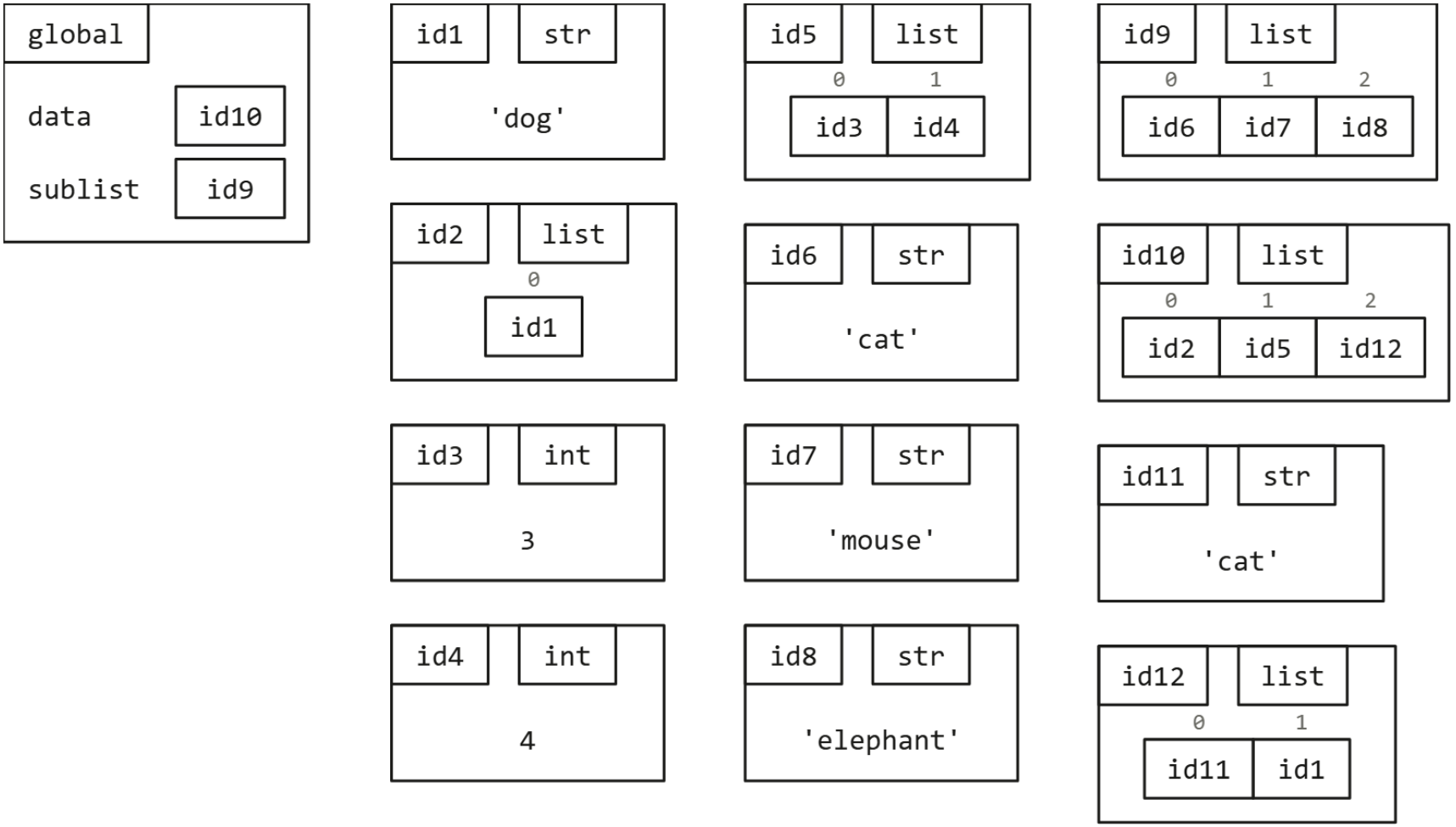
Program

```
1  data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
2  sublist = data[2]
3
4  data[2] = ['cat', data[0][0]]
5  print(data)
```

Shell

```
Python 3.11.3
```

Memory model (step 4)


global
data
id10
sublist
id9
id1
str
'dog'
id2
list
0
id1
id3
int
3
id4
int
4
id5
list
0
1
id3
id4
id6
str
'cat'
id7
str
'mouse'
id8
str
'elephant'
id9
list
0
1
2
id6
id7
id8
id10
list
0
1
2
id2
id5
id12
id11
str
'cat'
id12
list
0
1
id11
id1

main.py

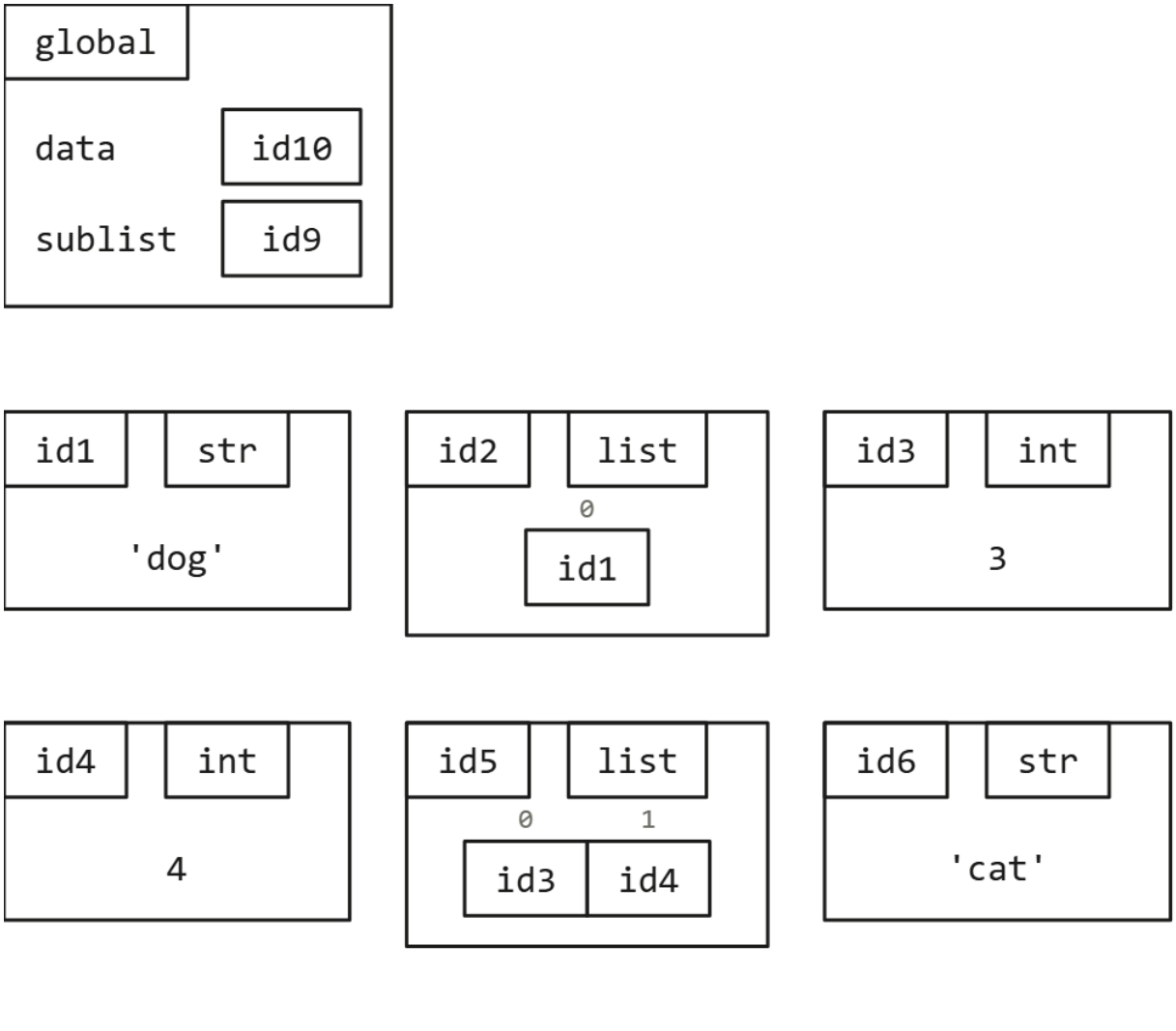
Program

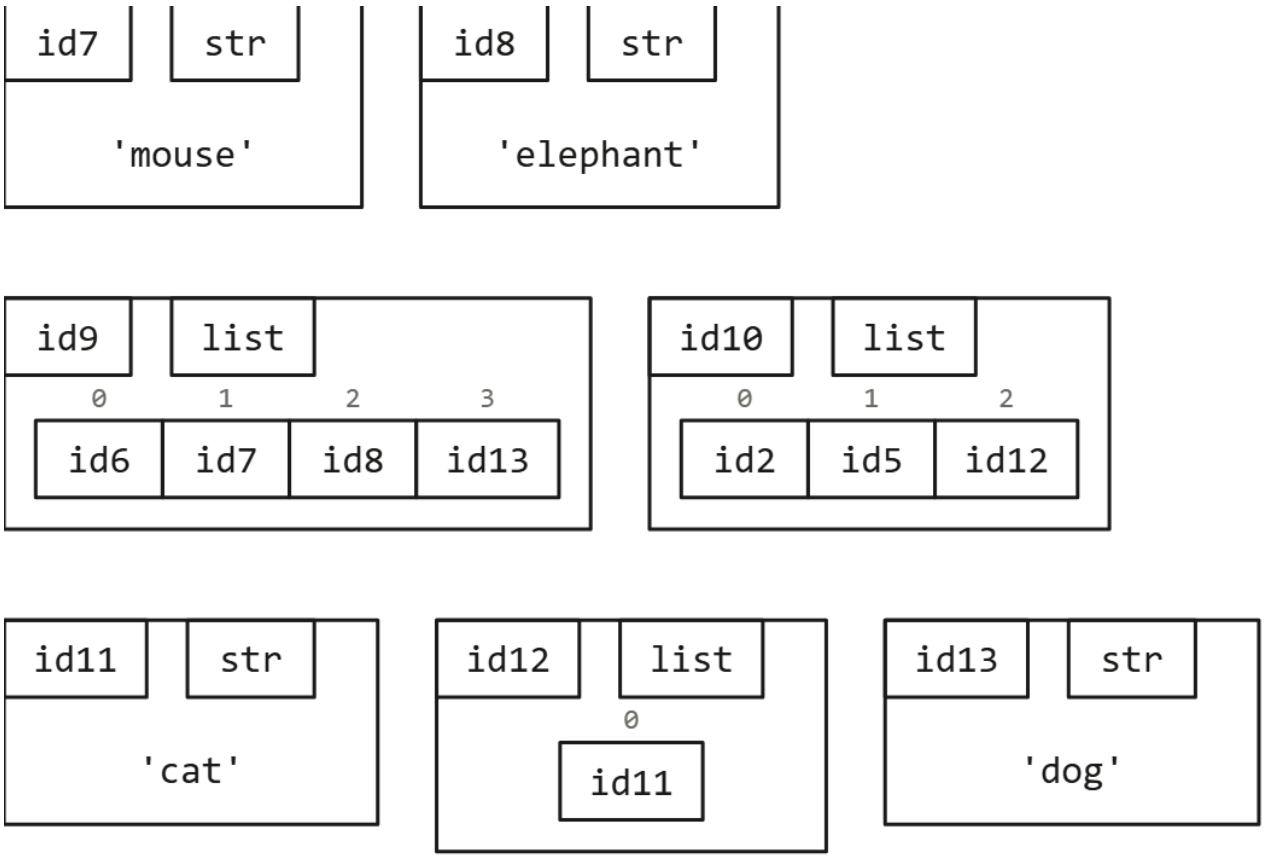
```
1  data = [['dog'], [3, 4], ['cat', 'mouse', 'elephant']]
2  sublist = data[2]
3
4  data[2] = ['cat']; sublist.append('dog')
5  print(data)
```

Shell

```
Python 3.11.3
Python 3.11.3
```

Memory model (step 5)


global
data
id10
sublist
id9
id1
str
'dog'
id2
list
0
id1
id3
int
3
id4
int
4
id5
list
0
1
id3
id4
id6
str
'cat'


id7
str
'mouse'
id8
str
'elephant'
id9
list
0
1
2
3
id6
id7
id8
id13
id10
list
0
1
2
id2
id5
id12
id11
str
'cat'
id12
list
0
id11
id13
str
'dog'

A `for` loop over the outer `list` gives the loop variable each **inner list** in turn.

```
>>> grid = [[1, 2, 3], [4, 5]]
>>> for row in grid:
...     print(row)
[1, 2, 3]
[4, 5]
```

The loop variable refers to an inner `list`, so indexing it picks out that inner `list`'s items.

```
>>> shopping = [['eggs', 12], ['apples', 6]]
>>> for item in shopping:
...     print(item[1], item[0])
12 eggs
6 apples
```

each inner `list` of `shopping` is `[item, quantity]`

calculate_areas.ipynb

Run the cell with `triangle_area` first. Then complete the examples in the docstring and the function body.

In [ ]

```
def triangle_area(base: float, height: float) -> float:
    """Return the area of the triangle with base and height.

    >>> triangle_area(10.0, 8.0)
    40.0
    """
    return base * height / 2
```

In [ ]

```python
def calculate_areas(triangles: list[list[float]]) -> list[float]:
    """Return a list of the areas of the triangles in triangles.
    Each inner list of triangles is [base, height].

    >>> calculate_areas([[10, 8], [5, 5], [7.5, 7.5]])
    [40.0, 12.5, 28.125]
    >>> calculate_areas([[4, 3], [6, 1]])
    [6.0, 3.0]
    """
    areas = []
    for triangle in triangles:
        areas.append(triangle_area(triangle[0], triangle[1]))
    return areas
```

In [8]

```python
calculate_areas([[10, 8], [5, 5], [7.5, 7.5]])
```

```
[40.0, 12.5, 28.125]
```

The loop variable refers to the inner `list` itself, so **changing** it changes the outer `list` too.

```
>>> playlists = [['jazz', 'pop'], ['rock']]
>>> for playlist in playlists:
...     playlist.append('lofi')
>>> playlists
[['jazz', 'pop', 'lofi'], ['rock', 'lofi']]
```

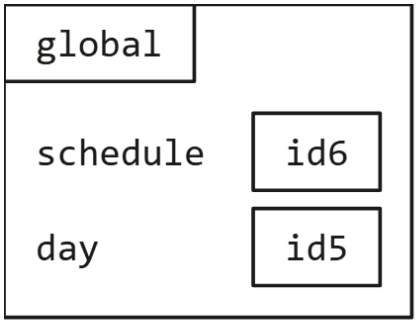
each `playlist` refers to an inner `list` of `playlists`

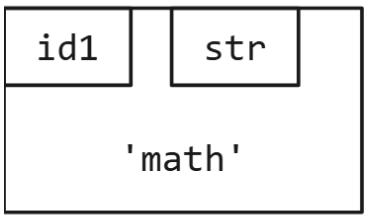
schedule.py

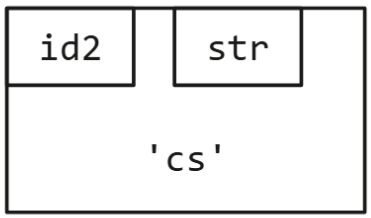
Program

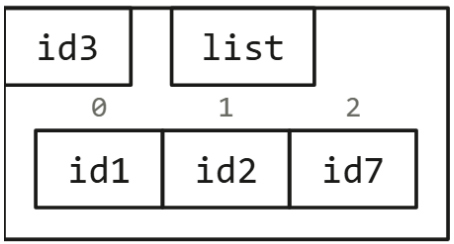
```
1  schedule = [['math', 'cs'], ['bio']]
2  for day in schedule:
3      day.append('lunch')
```

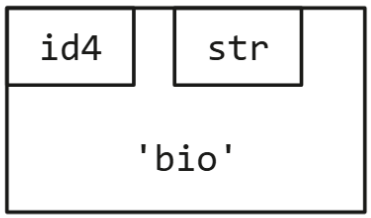
Memory model (step 5)


global
schedule
id6
day
id5

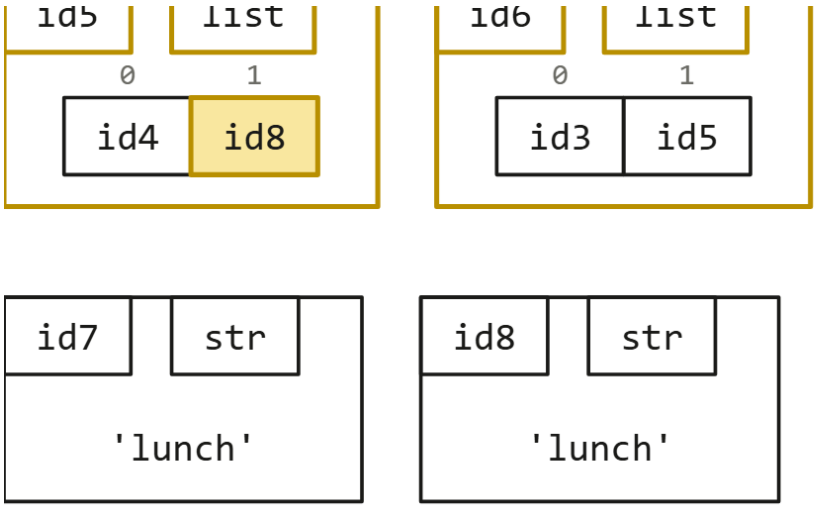

id1
str
'math'


id2
str
'cs'


id3
list
0
1
2
id1
id2
id7


id4
str
'bio'




id5
list
0
1
id4
id8
id6
list
0
1
id3
id5
id7
str
'lunch'
id8
str
'lunch'

swap_all_names.ipynb

Run the cell with `swap_name` first. Then complete the examples in the docstring and the function body.

In [ ]

```
def swap_name(name: list[str]) -> None:
    """Modify name so that its first and last items are swapped.

    >>> name = ['Grace', 'Hopper']
    >>> swap_name(name)
    >>> name
    ['Hopper', 'Grace']
    """
    first = name[0]
    name[0] = name[-1]
    name[-1] = first
```

In [ ]

```
def swap_all_names(names: list[list[str]]) -> None:
    """Modify names so that the first and last items of each inner list
    are swapped. Use swap_name as a helper.

    >>> scientists = [['Katherine', 'Johnson'], ['Geoffrey', 'Everest', 'Hinton']]
    >>> swap_all_names(scientists)
    >>> scientists
    [['Johnson', 'Katherine'], ['Hinton', 'Everest', 'Geoffrey']]
    >>> names = [['Ada', 'Lovelace'], ['Alan', 'Mathison', 'Turing'], ['Grace', 'Hopper']]
    >>> swap_all_names(names)
    >>> names
    ???
    """
```

In [ ]

```
scientists = [['Katherine', 'Johnson'], ['Geoffrey', 'Everest', 'Hinton']]
swap_all_names(scientists)
scientists
```

To reach each item, a second loop goes over each inner `list`. A loop inside another loop is a **nested loop**.

```
>>> grid = [[1, 2, 3], [4, 5]]
>>> for row in grid:
...     for num in row:
...         print(num)
1
2
3
4
5
```

inner loop: runs completely for each `row`

A nested loop can build up an **accumulator**, just like a single loop.

```
grid = [[1, 2, 3], [4, 5]]
total = 0
for row in grid:
    for num in row:
        total = total + num
# total is 15
```

row	num	total
[1, 2, 3]	1	1
	2	3
	3	6
[4, 5]	4	10
	5	15

digital_sum.ipynb

Complete the examples in the docstring and then the function body.

In [ ]

```
def digital_sum(nums_list: list[str]) -> int:
    """Return the sum of all the digits in all strings in nums_list.

    Precondition: s.isdigit() holds for each string s in nums_list.

    >>> digital_sum(['64', '128', '256'])
    34
    >>> digital_sum(['12', '3'])
    6
    """

    total = 0
    for num in nums_list:
        for ch in num:
            total = total + int(ch)

    return total
```

In [2]

```
digital_sum(['64', '128', '256'])
```

```
34
```

An accumulator set up **inside** the outer loop starts over for each inner `list`.

```
def total_marks(marks: list[list[int]]) -> list[int]:
    """Return a list of the total of each student's marks in marks.

    >>> total_marks([[8, 9, 7], [10, 6]])
    [24, 16]
    """
    totals = []
    for student in marks:
        total = 0
        for mark in student:
            total = total + mark
        totals.append(total)
    return totals
```

`total` goes back to `0` for each `student`

The inner loop does not have to go over an inner `list`. Nested loops over two `lists` pair every item of one with every item of the other.

```
>>> for n in [1, 2]:
...     for ch in ['a', 'b', 'c']:
...         print(n, ch)
1 a
1 b
1 c
2 a
2 b
2 c
```

pairs.py

Program

```
1  sizes = ['S', 'M', 'L']
2  colours = ['red', 'blue']
3
4  for size in sizes:
5      for colour in colours:
6          print(size, colour)
```

can_pay_with_two_coins.ipynb

Complete the examples in the docstring and then the function body.

In [ ]

```
def can_pay_with_two_coins(denoms: list[int], amount: int) -> bool:
    """Return True if and only if it is possible to form amount, which is a
    number of cents, using exactly two coins, which can be of any of the
    denominations in denoms.

    >>> can_pay_with_two_coins([1, 5, 10, 25], 35)
    True
    >>> can_pay_with_two_coins([1, 5, 10, 25], 20)
    True
    >>> can_pay_with_two_coins([1, 5, 10, 25], 12)
    ???
    """
```

In [ ]

```
can_pay_with_two_coins([1, 5, 10, 25], 35)
```